

ServerKit

Dev 2 — Interactive Shell + AI Layer

Your half: shell/, ai/ · depends on Dev 1's SDK

Your Role

You are building the two user-facing layers of ServerKit: the interactive shell that lets people explore the SDK, and the AI layer that lets them use natural language to drive it.

Important: You consume Dev 1's SDK. You do not reimplement it. Your code calls `server.processes()`, `server.logs()`, `server.workflow()` — it never calls `psutil` directly or reads files itself.

Your Modules

Module	Responsibility
shell/repl.py	The interactive REPL — reads input, calls SDK, prints output
shell/autocomplete.py	SDK-aware tab completion — deterministic, no AI
shell/parser.py	Parses shell input strings into SDK calls
ai/ollama_client.py	HTTP wrapper for the local Ollama API
ai/analyzer.py	Collects context, builds prompts, interprets responses

Working Before Dev 1 Finishes

You do not need to wait for Dev 1 to complete. Build against the agreed interface using stubs:

```
# stubs/server_stub.py – delete when Dev 1 is ready

class ProcessStub:
    def __init__(self, name, memory_mb, cpu_percent):
        self.name = name
        self.memory_mb = memory_mb
        self.cpu_percent = cpu_percent
        self.pid = 1234

class ProcessCollectionStub:
    def __init__(self):
        self.data = [
            ProcessStub('python', 1200, 12.0),
            ProcessStub('postgres', 800, 4.0),
            ProcessStub('nginx', 120, 0.5),
        ]
        def memory_above(self, mb):
            self.data = [p for p in self.data if p.memory_mb > mb]; return self
```

```
def all(self): return self.data
def summarize(self): return '\n'.join(f'{p.name}: {p.memory_mb}MB'
for p in self.data)

class ServerStub:
    def processes(self): return ProcessCollectionStub()
    def logs(self, path): return LogFileStub(path)
    def workflow(self, name): return WorkflowBuilderStub(name)
```

Interactive Shell

Purpose and Constraints

- The shell is a thin frontend for the SDK
- It does NOT implement business logic — it calls SDK methods and prints results
- It does NOT use AI for autocomplete — completions come from SDK metadata
- It does NOT replace bash — it only exposes SDK operations

shell/repl.py — The REPL

```
from prompt_toolkit import PromptSession
from prompt_toolkit.completion import Completer, Completion
from serverkit import Server
from shell.parser import parse_input
from shell.autocomplete import SDKCompleter

def run_shell():
    server = Server()
    session = PromptSession(completer=SDKCompleter())

    print('ServerKit Shell v1.0')
    print('Type help for commands. Ctrl+C to exit.')
    print()

    while True:
        try:
            text = session.prompt('> ')
            if not text.strip():
                continue
            if text.strip() == 'exit':
                break
            result = parse_input(text, server)
            if result is not None:
                print(result)
                print()
```

```
except KeyboardInterrupt:
    break
except Exception as e:
    print(f'Error: {e}')
```

shell/parser.py — Input Parser

The parser translates shell input strings into SDK calls. Keep it simple — you are not building a language parser, just pattern matching common commands.

```
def parse_input(text: str, server) -> str:
    text = text.strip()

    # processes.all()
    if text == 'processes.all()':
        procs = server.processes().all()
        return format_processes(procs)

    # processes.memory_above(N)
    if text.startswith('processes.memory_above('):
        n = extract_number(text)
        procs = server.processes().memory_above(n).all()
        return format_processes(procs)

    # processes.cpu_above(N)
    if text.startswith('processes.cpu_above('):
        n = extract_number(text)
        procs = server.processes().cpu_above(n).all()
        return format_processes(procs)

    # logs('path').errors()
    if 'logs(' in text and '.errors()' in text:
        path = extract_string_arg(text, 'logs')
        return server.logs(path).errors().summarize()

    # help
    if text == 'help':
        return HELP_TEXT

    return f'Unknown command: {text}'

def extract_number(text: str) -> float:
    import re
    match = re.search(r'\((\d+\.\d*)\)', text)
    return float(match.group(1)) if match else 0
```

```
def format_processes(procs: list) -> str:
    if not procs:
        return 'No processes found.'
    lines = [f'{p.name:<20} {p.memory_mb:>8.1f} MB    CPU:
{p.cpu_percent:.1f}%'
              for p in procs]
    return '\n'.join(lines)
```

shell/autocomplete.py — SDK-Aware Completion

Completions come from a static dictionary of SDK methods — not from AI, not from introspection. This is intentional. It is fast, reliable, and works offline.

```
from prompt_toolkit.completion import Completer, Completion

SDK_COMPLETIONS = {
    '': ['processes', 'logs', 'workflow', 'run', 'help', 'exit'],
    'processes': ['processes.all()', 'processes.named(',
'processes.memory_above(',
                  'processes.cpu_above(', 'processes.sort_by_memory()',
                  'processes.sort_by_cpu()'],
    'logs':      ['logs("app.log").errors()',
'logs("app.log").warnings()',
                  'logs("app.log").contains(', 'logs("app.log").tail(',
                  'logs("app.log").summarize()'],
    'workflow': ['workflow create ', 'workflow list', 'workflow run '],
}

class SDKCompleter(Completer):
    def get_completions(self, document, complete_event):
        text = document.text_before_cursor

        # Determine which prefix group to use
        prefix = ''
        for key in SDK_COMPLETIONS:
            if key and text.startswith(key):
                prefix = key
                break

        candidates = SDK_COMPLETIONS.get(prefix, SDK_COMPLETIONS[''])

        for candidate in candidates:
            if candidate.startswith(text):
                yield Completion(candidate, start_position=-len(text))
```

Shell Workflow Commands

The shell also supports interactive workflow creation — no AI involved:

```
# In parser.py — add workflow handling

if text.startswith('workflow create '):
    name = text.replace('workflow create ', '').strip()
    return run_workflow_builder(name, server)

if text == 'workflow list':
    workflows = server._workflow_manager.list()
    return '\n'.join(workflows) if workflows else 'No workflows saved.'

if text.startswith('workflow run '):
    name = text.replace('workflow run ', '').strip()
    server.run(name)
    return ''

def run_workflow_builder(name, server):
    print(f'Workflow Builder: {name}')
    print('Enter steps. Type save when done, cancel to abort.')
    builder = server.workflow(name)
    while True:
        step = input('step> ').strip()
        if step == 'save':
            builder.save()
            return f'Workflow saved: {name}'
        if step == 'cancel':
            return 'Cancelled.'
        apply_step_command(builder, step)
```

AI Layer

Philosophy

- AI translates natural language into SDK actions — it never executes arbitrary commands
- AI never calls psutil, reads files, or runs shell commands directly
- The SDK is always in control of execution
- All AI functionality is optional — the SDK and shell work without it

ai/ollama_client.py — Ollama Wrapper

```
import requests
import json

OLLAMA_URL = 'http://localhost:11434/api/generate'
DEFAULT_MODEL = 'phi3' # or 'qwen2.5:1.5b', 'smollm2'

class OllamaClient:
    def __init__(self, model=DEFAULT_MODEL):
        self.model = model

    def ask(self, prompt: str) -> str:
        payload = {
            'model': self.model,
            'prompt': prompt,
            'stream': False
        }
        try:
            resp = requests.post(OLLAMA_URL, json=payload, timeout=30)
            resp.raise_for_status()
            return resp.json()['response']
        except requests.ConnectionError:
            raise RuntimeError('Ollama not running. Start with: ollama serve')
        except Exception as e:
            raise RuntimeError(f'Ollama error: {e}')

    def is_available(self) -> bool:
        try:
            requests.get('http://localhost:11434', timeout=2)
            return True
        except:
            return False
```

ai/analyzer.py — The Analyzer

The Analyzer is responsible for three things: intent-to-action translation, diagnostic analysis, and workflow generation from natural language.

```
import json
from ai.ollama_client import OllamaClient

class Analyzer:
    def __init__(self, server, model='phi3'):
        self.server = server
        self.ollama = OllamaClient(model)
```

```

def ask(self, query: str):
    # Decide mode based on query
    if 'create a workflow' in query.lower() or 'make a workflow' in query.lower():
        return self._generate_workflow(query)
    elif any(w in query.lower() for w in ['why', 'diagnose', 'what is causing']):
        return self._diagnose(query)
    else:
        return self._execute_intent(query)

```

Intent → SDK Action

```

def _execute_intent(self, query: str):
    prompt = f'''
You are a Linux server assistant. Convert the user query into a JSON SDK action.
Return ONLY valid JSON, no explanation.

```

Available resources: processes, logs

Available actions: memory_above, cpu_above, named, errors, warnings, contains, tail, summarize

Examples:

Query: show python processes using more than 1GB RAM

JSON: [{"resource": "processes", "filters": [{"action": "named", "value": "python"}], [{"action": "memory_above", "value": 1000}]}]

Query: {query}

JSON:'''

```

    raw = self.ollama.ask(prompt)
    try:
        action = json.loads(raw.strip())
        return self._run_action(action)
    except json.JSONDecodeError:
        return f'Could not parse response: {raw}'

def _run_action(self, action: dict):
    resource = action.get('resource')
    if resource == 'processes':
        collection = self.server.processes()
        for f in action.get('filters', []):
            if f['action'] == 'memory_above':
                collection = collection.memory_above(f['value'])
            elif f['action'] == 'cpu_above':
                collection = collection.cpu_above(f['value'])
            elif f['action'] == 'named':
                collection = collection.named(f['value'])

```



```
        return collection.summarize()
    return 'Unsupported resource'
```

Diagnostic Mode

```
def _diagnose(self, query: str):
    # 1. Collect context from SDK
    procs = self.server.processes().sort_by_memory().all()[0:10]
    proc_summary = '\n'.join(
        f'{p.name}: {p.memory_mb:.0f}MB, CPU {p.cpu_percent:.1f}%'
        for p in procs
    )

    # 2. Build prompt with context
    prompt = f'''
You are a Linux server diagnostician.
Top 10 processes by memory:
{proc_summary}

User question: {query}

Give a clear, concise diagnosis in 3-5 lines.
'''

    # 3. Ask model, return response
    return self.ollama.ask(prompt)
```

AI Workflow Generation

```
def _generate_workflow(self, query: str):
    prompt = f'''
Convert this request into a ServerKit workflow JSON.
Return ONLY valid JSON matching this schema exactly:
{{
  "name": "workflow_name",
  "steps": [
    {{ "type": "process_filter", "memory_above": 1000 }},
    {{ "type": "sort", "field": "memory" }},
    {{ "type": "summary" }}
  ]
}}

Step types: process_filter, sort, log_filter, tail, summary, export

Request: {query}
JSON: '''
```

```
raw = self.ollama.ask(prompt)
try:
    wf_data = json.loads(raw.strip())
    # Hand off to Dev 1's Workflow.from_dict()
    from serverkit.workflows.workflow import Workflow
    wf = Workflow.from_dict(wf_data)
    wf.save()
    return f'Workflow created and saved: {wf.name}'
except Exception as e:
    return f'Failed to generate workflow: {e}\nRaw: {raw}'
```

Integrating AI into Server

The Analyzer gets wired into the Server class by Dev 1 (or you can propose this addition):

```
# In core/server.py – add this method:

def ask(self, query: str) -> str:
    from ai.analyzer import Analyzer
    analyzer = Analyzer(self)
    return analyzer.ask(query)
```

Note: Import is inside the method so the AI layer stays optional. If Ollama is not installed, the rest of the SDK still works fine.

Prompt Engineering Tips

Working with small local models (Phi-3, Qwen2.5) requires careful prompting. These models are capable but need clear constraints.

Tip	Why
Always ask for JSON only	Add 'Return ONLY valid JSON, no explanation' to every structured prompt
Give examples in prompt	One or two JSON examples dramatically improve output quality
Keep prompts short	Small models perform worse with very long prompts — be concise

Parse defensively	Always wrap <code>json.loads()</code> in <code>try/except</code> — models occasionally output markdown fences
Strip the response	<code>raw.strip()</code> removes leading/trailing whitespace that breaks JSON parsing
Log failures	Print the raw response when parsing fails — helps you improve the prompt

Shared Contracts with Dev 1

These are the methods you call from Dev 1's SDK. Do not bypass them.

Method	What You Do With It
<code>server.processes()</code>	Returns <code>ProcessCollection</code> — you chain filters on this
<code>server.logs(path)</code>	Returns <code>LogFile</code> — you chain <code>.errors()</code> <code>.tail()</code> etc.
<code>server.workflow(name)</code>	Returns <code>WorkflowBuilder</code> — you call <code>.save()</code> at the end
<code>server.run(name)</code>	Runs a saved workflow by name
<code>Workflow.from_dict(data)</code>	Reconstructs a <code>Workflow</code> from the JSON your AI generates
<code>.summarize()</code>	Returns <code>str</code> — the terminal method you print in the shell
<code>.all()</code>	Returns <code>list</code> — use when you need to iterate or count

Suggested Build Order

Milestone	Acceptance Test
1. OllamaClient	Test: <code>client.is_available()</code> , <code>client.ask('say hello')</code> returns a string
2. Shell REPL (stub server)	Test: shell launches, <code>processes.all()</code> prints stub data, exit works
3. Autocomplete	Test: TAB after 'proc' completes to 'processes', TAB after 'processes.' shows options
4. Parser — process commands	Test: <code>processes.memory_above(500)</code> calls stub and prints formatted output
5. Parser — logs commands	Test: <code>logs('app.log').errors()</code> runs without error
6. Analyzer — intent mode	Test: <code>ask('show high memory processes')</code> returns formatted process list

7. Analyzer — diagnostic mode	Test: ask('why is memory high') returns a paragraph from Ollama
8. Analyzer — workflow generation	Test: ask('create a memory audit workflow') saves a .json file
9. Connect to real SDK	Swap stubs for real server from Dev 1. Retest all above.

Tip: Steps 1-8 can be done entirely with stubs. Only step 9 requires Dev 1's code to be ready. Plan so you finish step 8 before Dev 1 finishes, then integrate quickly.